Assignment 3

🤖 Artificial Intelligence for Robotics

Given a tiny 2-D simulator in Canvas (`wf_sim.py`). A differential-drive robot drives forward at a constant speed alongside a wall (the line y=0) and must hold a target distance d^* using a noisy distance sensor. Everything happens in a discrete sense → decide → act loop. The starter code produces two plots and prints a **metrics** table. Each run is scored by a small utility/cost: RMS tracking error, max overshoot, control effort ($\sum|\omega| \cdot \Delta t$), and % deadline misses.
You will turn this toy into a real control study: implement PID, break it with timing, make it open-world-safe, and finally let it tune itself under a compute budget. The fun is in watching it oscillate, lag, stutter, and then behave.

1. [Orientation, 10 pts] Run the starter as-is. In your report, briefly answer: with the given proportional controller, why does a higher K_p make the robot oscillate instead of settling? Connect this to slide 8 (gain too small vs. too large) and to the fact that here $\omega = K_p \cdot e$ sets the heading rate from a position error (so error feeds through an extra integration before it shows up in y)

2. [Characterize proportional control, 20 pts] Sweep K_p over at least five values (e.g., 0.2, 0.5, 1.5, 4.0, 8.0). For each, record the four metrics and plot the tracking error vs. time on one figure.
   (a) Produce a table of K_p vs. {RMS, overshoot, effort}.
   (b) Identify the qualitative regimes — sluggish, reasonable, oscillatory — and the approximate K_p where each begins.
   (c) Explain why no single K_p makes this plant settle cleanly, motivating the need for the D term.

3. [Implement PID and tune it, 20 pts] Fill in the `PIDController` TODOs (the P term is given; add I and D). Then tune (K_p, K_i, K_d) for the setpoint-step scenario already wired into `main()` (d^* jumps from 1.0 → 1.5 m at t = 20 s).
   (a) Working PID implementation that handles the first step (no previous error) safely.
   (b) A figure showing trajectory + error for your tuned PID through the step, plus the metrics. Target: settled RMS error < 0.05 m after each transient.
   (c) Ablation: show the metrics for P-only, PD, and full PID on the same scenario, and explain in 3–4 sentences what each added term bought you (steady-state offset? damping?).

4. [Break it with timing, 20 pts] Real loops are not instant or perfectly periodic. Using your tuned PID, study three effects independently:
   (a) Sampling rate. Vary `dt_nominal` (e.g., 100 Hz, 20 Hz, 5 Hz, 2 Hz). Plot/record how RMS and overshoot change. At what rate does control visibly fall apart, and why?

(b) Latency. Sweep `latency_steps` (the controller acts on a delayed measurement). Show the degradation and explain why delay specifically destabilizes feedback (the correction is fighting a stale error).

(c) Jitter. Set `jitter_std` and watch `deadline_miss%` climb. The deck leaves this open (slide 14): propose a concrete, quantitative criterion for when jitter is "problematic" — not "it looks noisy." Apply it to your runs and connect it to the real-time condition $T_{loop} < T_{environmen_change}$ (slide 46). Then classify which of the robot's behaviors (route update, motor control, display, emergency stop) are soft vs. hard real time and say which one jitter endangers most.

5. [Make it open-world-safe, 20 pts] The wall-follower assumes a clear path — a closed-world assumption. Set `obstacle_x` so a blockage crosses the path.

   (a) First run without any override and document the functional failure: the controller code is correct, but the robot drives into the obstacle. Capture the trajectory.

   (b) Implement the Part 4 TODO reactive safety override in `run_sim` using `front_obstacle_distance(...)` and `cfg.SAFE_DIST`: when the path is blocked, stop/steer away and set overridden = True. This is a bounded action space + safe fallback layered over the feedback controller. Show the robot now avoids the blockage.

   (c) Argue in 4–6 sentences why adding feedback was necessary but not sufficient for open-world safety, and which Module-02 architecture layer this override belongs to (reactive vs. deliberative) and why.

6. [Bounded-rationality autotuner, 10 pts] A perfectly rational agent would find the optimal gains over all possibilities; a bounded-rational one searches the feasible set within a time/compute budget.

   Write a function autotune(cfg, budget) that grid-searches (K_p, K_i, K_d) over a bounded set, runs the sim for each, and returns the gains minimizing a cost you define from the metrics (e.g., RMS + 0.1·effort, your choice — this is the "utility," slide 37). Cap the number of evaluated candidates at budget and stop when you hit it.

   (a) Report the best gains found and their metrics.

   (b) Run the autotuner with a small budget and a large budget. Show that the small-budget result is worse, and connect this directly to the formula $\hat{a} = argmax_a \in A_{feasible}\ \mathbb{E}[U \mid a, time\ limit]$ vs. $a* = argmax_a\ \mathbb{E}[U \mid a, all\ info]$ (slide 40). One paragraph.

   Reflection (bundled into the parts above)

   Somewhere in your report, tie at least three of these course ideas to something you observed in code: feedback/error correction, soft vs. hard real time, closed- vs. open-world, signals-vs-symbols, bounded rationality, the substitution myth, sense-plan-act vs. OODA. Be concrete ("when I set latency to 15 steps, …"), not generic.